Contenido

Práctica 3 Configuración HTTPS mediante certificado SSL/TLS autofirmado 2
Objetivos 2
Relación con los RA 2
Antes de empezar 3
Actividad 1. Comprobar el estado actual 3
Actividad 2. Comprobar que OpenSSL está instalado 3
Actividad 3. Crear directorio para certificados 3
Actividad 4. Generar el certificado SSL y rellenar los datos 4
Actividad 6. Activar SSL en Apache 5
Actividad 7. Crear Virtual Host HTTPS 6
Actividad 8. Habilitar el sitio HTTPS 7
Actividad 9. Verificar puerto 443 7
Actividad 10. Acceder mediante HTTPS 8
Actividad 11. Ver información del certificado 10
Actividad 12. Comparación HTTP vs HTTPS 11
Conclusiones 11

Práctica 3 Configuración HTTPS mediante certificado SSL/TLS autofirmado

Objetivos

Actualmente disponemos de los dominios ficticios creados en la Práctica 2:

- http://empresa1.local
- http://empresa2.local

funcionando sobre el **Puerto 80.** Ahora queremos llegar a que https://empresa1.local utilizando el **Puerto 443**

El servicio principal que se emplea con el **puerto 443 es el HTTPS** (Hypertext Transfer Protocol Secure o Protocolo Seguro de Transferencia de Hipertexto). Es la versión segura y cifrada del protocolo **HTTP tradicional (que usa el puerto 80)**. Se utiliza para la navegación web segura, garantizando que toda la información transmitida entre el navegador del usuario y el servidor web viaje protegida.

Para que el **puerto 443** funcione de forma segura, **el servidor necesita** utilizar un **certificado digital** basado en los siguientes **protocolos de cifrado TLS** (Transport Layer Security) o su predecesor **SSL** (Secure Sockets Layer).

Prácticamente cualquier sitio web moderno (redes sociales, bancos, tiendas online o tu propio sitio empresa1.local si decides añadirle seguridad) utiliza el **puerto 443**. Es el estándar actual para proteger:

- Contraseñas y datos de inicio de sesión.
- Datos de tarjetas de crédito y transacciones bancarias.
- Información médica o personal sensible.

Como queremos que nuestro sitio **empresa1.local** sea seguro (es decir, que se abra con https://), tenemos que crear un nuevo bloque de configuración en Apache que empiece apuntando a ese **puerto 433 generando un certificado autoafirmado gratis** para añadir seguridad HTTPS a nuestro entorno de pruebas en Apache.

Al finalizar la práctica el seremos capaces de:

- Comprender las diferencias entre HTTP y HTTPS.
- Crear un certificado SSL/TLS autofirmado.
- Activar HTTPS en Apache.
- Utilizar el puerto 443.
- Analizar la información del certificado.
- Acceder a una aplicación mediante HTTPS.

Relación con los RA

Con esta práctica cubrimos:

- **RA2.b** Configurar servicios relacionados con DNS y resolución de nombres.
- **RA2.c** Analizar diferencias entre HTTP y HTTPS.
- **RA2.e** Configurar nombres de dominio para aplicaciones web.

- **RA3.a** Configurar Apache para servir aplicaciones mediante HTTPS.

Antes de empezar

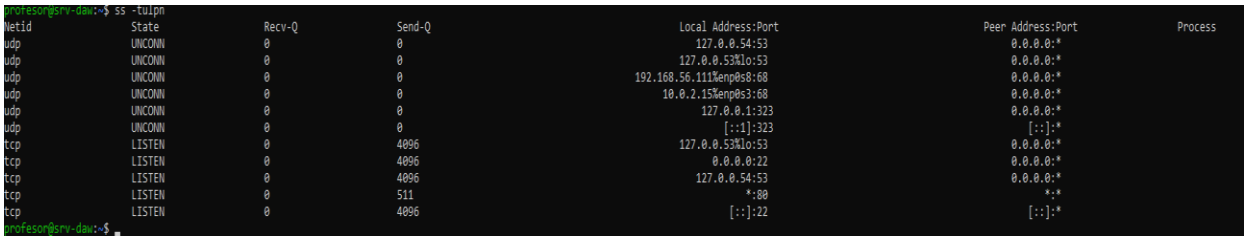
Como ya has conseguido que funcionen los Virtual Hosts, es recomendable hacer primero un snapshot: **LAB_U2_P2_VIRTUALHOST_OK** porque si la configuración SSL queda mal, podremos volver a este punto en segundos sin reconfigurar Apache ni los Virtual Hosts.

Actividad 1. Comprobar el estado actual

Verificar Apache: **sudo systemctl status apache2**



Comprobar puertos abiertos: **ss -tulpn**



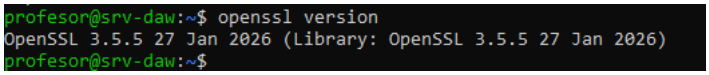
Actualmente debería aparecer: ***:80** y NO: ***:443**

¿Cuál es el problema de HTTP?

HTTP transmite la información sin cifrar. Por ejemplo usuario, contraseña, cookies, formularios etc. podrían ser interceptados.

Actividad 2. Comprobar que OpenSSL está instalado

Ejecutar: **openssl versión**



Debe devolver algo parecido a: **OpenSSL 3.x.x**

Si no estuviera instalado: sudo **apt install openssl -y**

Actividad 3. Crear directorio para certificados

sudo mkdir /etc/apache2/ssl

Comprobar: **ls /etc/apache2**



Actividad 4. Generar el certificado SSL y rellenar los datos

Para generar el **certificado SSL** vamos a ejecutar un comando que se utiliza para generar un **certificado SSL/TLS autoafirmado (self-signed) y su respectiva clave privada** utilizando la **herramienta OpenSSL**. Este tipo de certificados es ideal para entornos de desarrollo o pruebas locales:

**sudo openssl req -x509 -nodes -days 365 **

**-newkey rsa:2048 **

**-keyout /etc/apache2/ssl/empresa1.key **

-out /etc/apache2/ssl/empresa1.crt

- **openssl req**
  Invoca la herramienta OpenSSL e indica que se va a utilizar el módulo de administración de Solicitudes de Firma de Certificado (CSR) y generación de certificados.
- **-x509**
  Le dice a OpenSSL que cree un certificado autoafirmado en lugar de generar una simple solicitud (CSR) para enviar a una Entidad Certificadora (como Let's Encrypt o DigiCert).
- **-nodes**
  Se lee "no-DES" y significa que no se protegerá la clave privada con una contraseña. Esto es fundamental en servidores web como Apache; de lo contrario, cada vez que reiniciemos el servidor, este se quedaría pausado pidiéndote que escribamos la contraseña en la terminal.
- **-days 365**
  Establece el periodo de validez del certificado. En este caso, el certificado será válido durante un año (365 días) a partir de hoy. Pasado ese tiempo, caducará y tendremos que generar uno nuevo.
- **-newkey rsa:2048**
  Crea una nueva clave privada al mismo tiempo que el certificado. El algoritmo criptográfico utilizado será RSA con una longitud de 2048 bits, que es el estándar recomendado actualmente por su equilibrio entre seguridad y rendimiento**.**
- **-keyout /etc/apache2/ssl/empresa1.key**
  Especifica la ruta absoluta y el nombre del archivo donde se guardará la clave privada generada. Esta clave es secreta y nadie más que el servidor debe tener acceso a ella.
- **-out /etc/apache2/ssl/empresa1.crt**
  Especifica la ruta absoluta y el nombre del archivo donde se guardará el certificado público. Este es el archivo que el servidor Apache enviará a los navegadores web para identificarse.

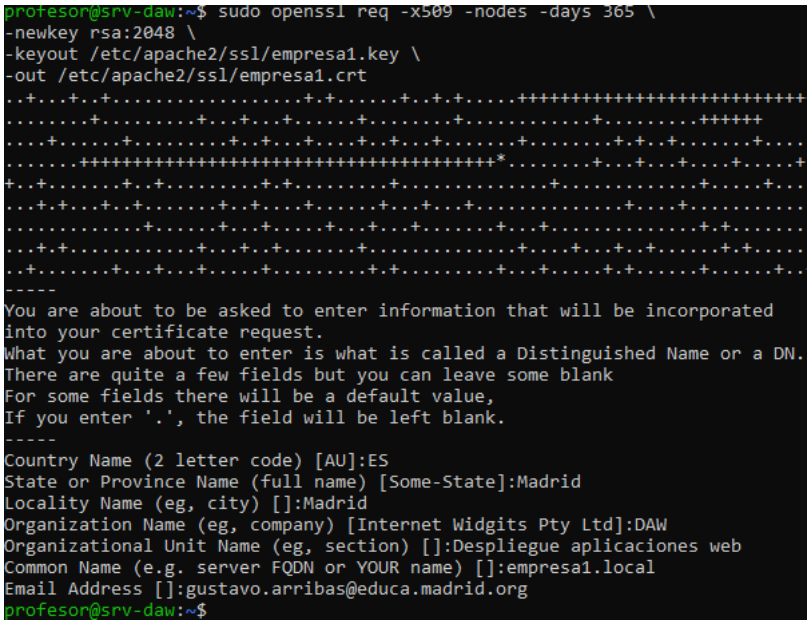
Por tanto al generar el **certificado SSL/TLS autoafirmado** se generarán a su vez dos ficheros:

- **Clave privada empresa1.key**
- **Certificado empresa1.crt**

Cuando presiones Enter, la terminal nos hará una serie de preguntas para rellenar los datos de identidad del certificado (País, Estado, Organización, etc.). Los valores recomendados son:

- Country Name: **ES**
- State: **Madrid**
- Locality: **Madrid**
- Organization: **DAW**
- Organization unit: **Despliegue de aplicaciones web**
- Common Name: **empresa1.local**
- Email Addres: **gustavo.arribas@educa.madrid.org**

La pregunta más importante es el **Common Name.** Ahí debemoss escribir exactamente el dominio de tu Virtual Host: **empresa1.local**. Si no coincide, el navegador mostrará un error de seguridad aún mayor.



Actividad 6. Activar SSL en Apache

Necesitamos activar el módulo **mod_ssl** en el servidor web Apache sobre sistemas operativos basados en Debian/Ubuntu.

Activar módulo: **sudo a2enmod ssl**

- **a2enmod:** Es un script específico de Apache en Debian y Ubuntu que sirve para habilitar un módulo. Crea un enlace simbólico del archivo de configuración del módulo desde el directorio de disponibles (/etc/apache2/mods-available/) hacia el directorio de habilitados (/etc/apache2/mods-enabled/).

- **ssl:** Es el nombre del módulo que se desea activar (mod_ssl), el cual proporciona soporte para el cifrado SSL/TLS y permite que Apache gestione conexiones seguras a través del protocolo HTTPS.

```
profesor@srv-daw:~$ sudo a2enmod ssl
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Enabling module socache_shmcb.
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and create self-signed certificates.
To activate the new configuration, you need to run:
  systemctl restart apache2
profesor@srv-daw:~$
```

Comprobar: **sudo apache2ctl -M | grep ssl**

Resultado esperado: ssl_module

```
profesor@srv-daw:~$ sudo apache2ctl -M | grep ssl
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress this message
 ssl_module (shared)
profesor@srv-daw:~$
```

Un detalle importante para entornos locales (.local) dado que estamos utilizando el dominio ficticio **empresa1.local,** si intentamos entrar desde el navegador de nuestra máquina habitual no cargará a menos que le digamos a nuestro sistema dónde encontrarlo. Como estamos probando en el mismo equipo donde corre Apache, debemos asegúranos de añadir la línea (192.168.56.111 empresa1.local) al archivo **/etc/hosts** que contiene la resolución de nombres en el servidor srv_daw de **Ubuntu** o el archivo de **hosts** del DNS local de Windows en **C:\Windows\System32\drivers\etc** para poder resolver el dominio **empresa1.local** en nuestro navegador de nuestro sistema operativo.

Actividad 7. Crear Virtual Host HTTPS

Vamos a crear un archivo que define un **Host Virtual (VirtualHost)** específico para gestionar el tráfico seguro bajo el protocolo **HTTPS**.

Crear: **sudo nano /etc/apache2/sites-available/empresa1-ssl.conf**

Contenido del fichero de configuración Apache:

<VirtualHost *:443>

ServerName empresa1.local

DocumentRoot /var/www/empresa1

SSLEngine on

SSLCertificateFile /etc/apache2/ssl/empresa1.crt

SSLCertificateKeyFile /etc/apache2/ssl/empresa1.key

</VirtualHost>

Guardar.

Cada línea del fichero de configuración cumple una función específica dentro de la directiva:

- **<VirtualHost *:443>**: Indica que esta configuración se aplicará a cualquier dirección IP (*) del servidor que reciba peticiones a través del puerto **443**, que es el puerto estándar mundial para el tráfico seguro **HTTPS**.
- **ServerName empresa1.local**: Define el nombre de dominio al que responderá este VirtualHost. El servidor solo usará esta configuración si la solicitud del navegador coincide exactamente con empresa1.local.
- **DocumentRoot /var/www/empresa1**: Especifica la ruta física del directorio en el servidor donde están guardados los archivos de la página web (los HTML, PHP, imágenes, etc.) que se mostrarán al usuario.
- **SSLEngine on**: Activa formalmente el motor de cifrado SSL/TLS para este Host Virtual específico. Sin esta línea, el puerto 443 intentaría responder en texto plano, rompiendo la comunicación.
- **SSLCertificateFile /etc/apache2/ssl/empresa1.crt**: Indica la ruta absoluta donde se encuentra el **certificado de clave pública** de tu sitio. Es el archivo que Apache envía al navegador del usuario para demostrar la identidad del sitio web.
- **SSLCertificateKeyFile /etc/apache2/ssl/empresa1.key**: Indica la ruta absoluta a la **clave privada** del certificado. Este archivo es crítico para el cifrado y **nunca debe ser público** ni compartirse, ya que permite al servidor descifrar los datos enviados por los usuarios.
- **</VirtualHost>**: Cierra el bloque de configuración del Host Virtual.

Una vez que hemos guardado el archivo en nano, debemoss indicarle a Apache que empiece a utilizarlo para lo cual ejecutaremos una serie comandos en nuestro terminal para habilitar el nuevo sitio virtual, verificar que la sintaxis no tenga errores y reiniciar Apache para aplicar los cambios.

Actividad 8. Habilitar el sitio HTTPS

Para habilitar el nuevo sitio virtual ejecutamos el comando:

sudo a2ensite empresa1-ssl.conf

```
profesor@srv-daw:~$ sudo a2ensite empresa1-ssl.conf
Enabling site empresa1-ssl.
To activate the new configuration, you need to run:
  systemctl reload apache2
profesor@srv-daw:~$
```

Para comprobar sintaxis no tenga errores: **sudo apache2ctl configtest**

```
profesor@srv-daw:~$ sudo apache2ctl configtest
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress this message
Syntax OK
```

Debe aparecer: **Syntax OK**

Para reiniciar o recargar Apache: **sudo systemctl reload apache2**

Actividad 9. Verificar puerto 443

Ejecutar: **ss -tulpn**

```
profesor@srv-daw:~$ ss -tulpn
Netid               State                Recv-Q               Send-Q                             Local Address:Port                             Peer Address:Port              Process
udp                 UNCONN               0                    0                                     127.0.0.54:53                                  0.0.0.0:*
udp                 UNCONN               0                    0                                  127.0.0.53%lo:53                                  0.0.0.0:*
udp                 UNCONN               0                    0                          192.168.56.111%enp0s8:68                                  0.0.0.0:*
udp                 UNCONN               0                    0                             10.0.2.15%enp0s3:68                                  0.0.0.0:*
udp                 UNCONN               0                    0                                   127.0.0.1:323                                   0.0.0.0:*
udp                 UNCONN               0                    0                                       [::1]:323                                      [::]:*
tcp                 LISTEN               0                    4096                                127.0.0.53%lo:53                                  0.0.0.0:*
tcp                 LISTEN               0                    4096                                     0.0.0.0:22                                   0.0.0.0:*
tcp                 LISTEN               0                    4096                                  127.0.0.54:53                                   0.0.0.0:*
tcp                 LISTEN               0                    511                                         *:443                                        *:*
tcp                 LISTEN               0                    511                                         *:80                                         *:*
tcp                 LISTEN               0                    4096                                     [::]:22                                      [::]:*
profesor@srv-daw:~$
```

Ahora debería aparecer: ***:80** y ***:443**

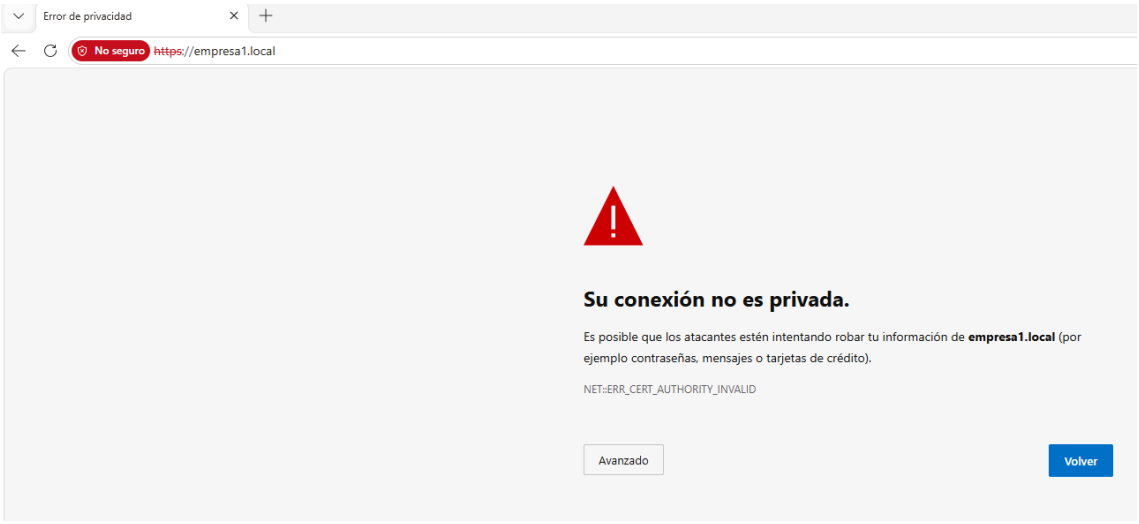
¿Qué significa que Apache escucha en el puerto 443?

Significa que Apache está preparado para aceptar conexiones HTTPS cifradas.

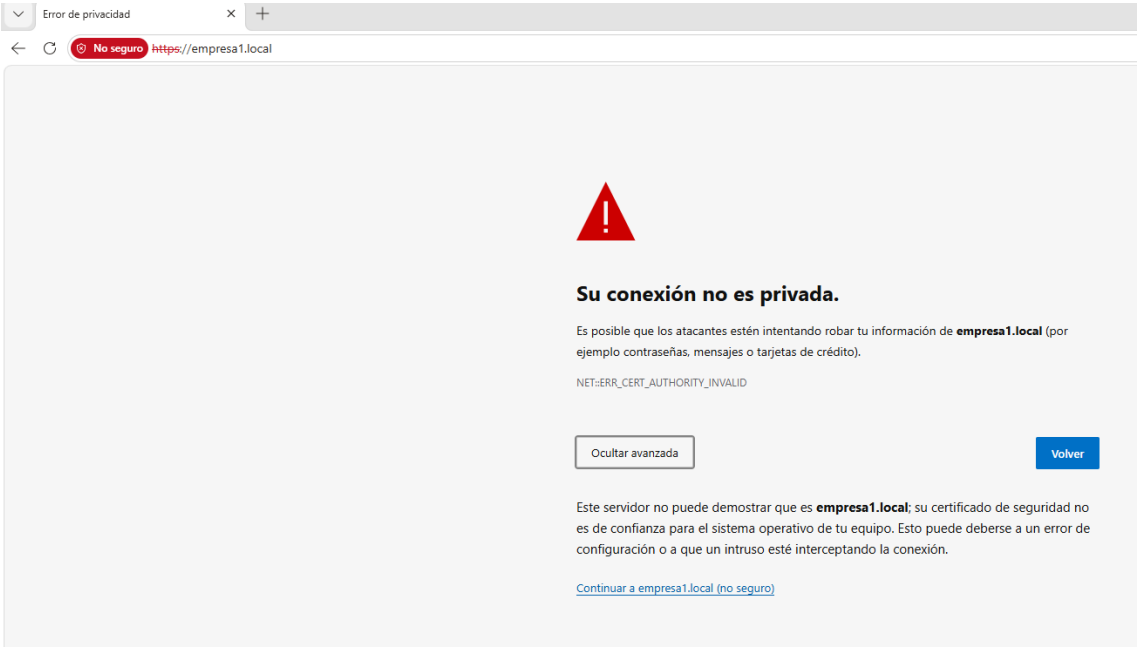
Actividad 10. Acceder mediante HTTPS

Desde Windows: **https://empresa1.local**

Probablemente aparezca una advertencia. Esto es normal.



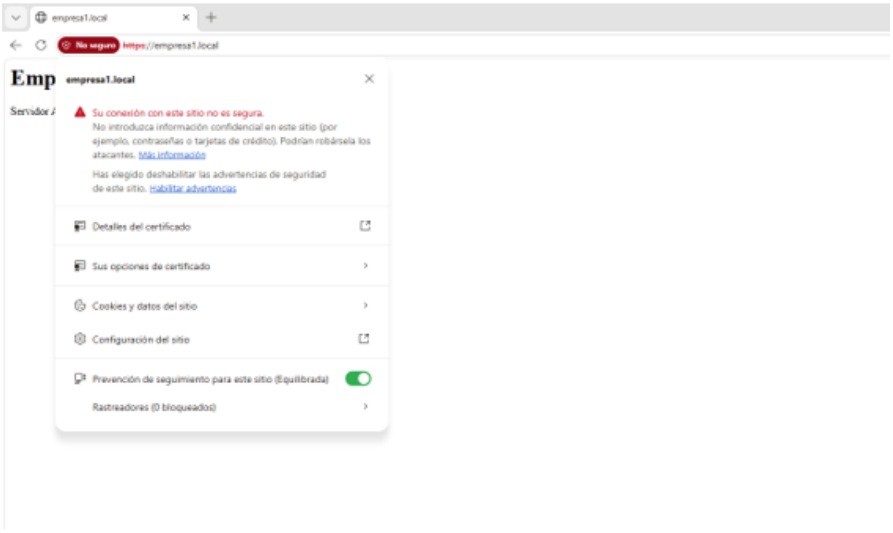
Pulsamos el botón de Avanzado



Pulsamos en Continuar a empresa1.local (no seguro)



Pulsamos en No seguro (En la barra del navegador en rojo)

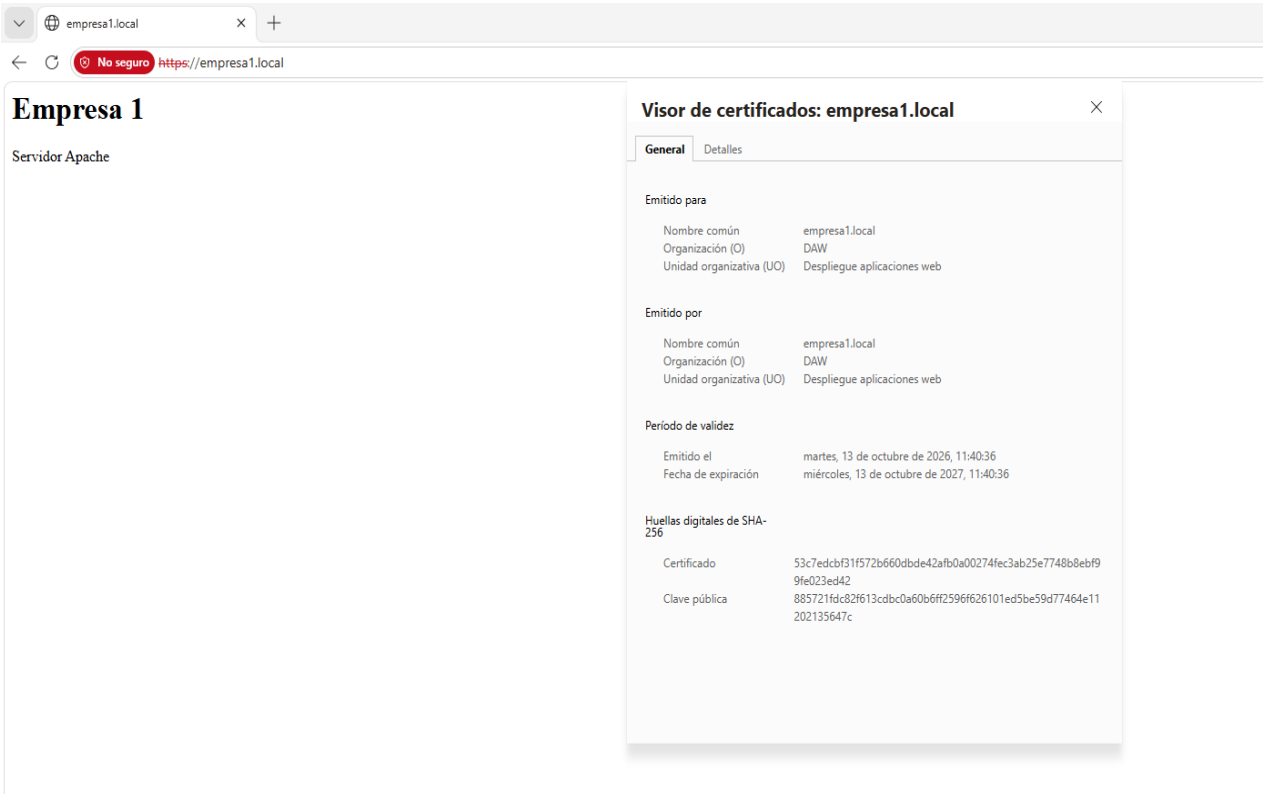


¿Por qué aparece el aviso?

Porque el certificado es Autofirmado. No está emitido por Let's Encrypt, DigiCert, GlobalSign o Sectigo.

Actividad 11. Ver información del certificado

Pulsar: **Ver certificado**



Comprobar: empresa1.local y validez

El certificado es autofirmado (self-signed certificate) y no ha sido emitido por una Autoridad de Certificación (CA) reconocida por el navegador. Es decir:

- Let's Encrypt Confiable
- DigiCert Confiable
- Sectigo Confiable
- empresa1.local Autofirmado

Por eso el navegador muestra No seguro aunque la comunicación sí está cifrada. El aviso que estás viendo en nuestro navegador se debe exactamente a la distinción entre **cifrado de datos** e **identidad del sitio**:

- **La comunicación sí está cifrada:** Al usar el protocolo HTTPS, toda la información que viaja entre nuestro navegador y el servidor Apache viaja de forma totalmente segura e inaccesible para terceros. El motor SSLEngine on que configuramos está haciendo su trabajo.
- **La identidad no está validada:** Los navegadores (como Chrome, Firefox o Edge) tienen una lista interna de Autoridades de Certificación (CA) de confianza mundial (como Let's Encrypt, DigiCert, etc.). Como nuestro certificado lo hemos generado nosotros mismo (autofirmado), el navegador no tiene ninguna forma independiente de verificar que nosotros somos realmente quien decimos ser, y por eso muestra la advertencia de **"No seguro"** o **"Conexión privada"**.

Para nuestras pruebas en un entorno de desarrollo local (.local), **este comportamiento es el esperado y es perfectamente seguro** aceptar el riesgo, por ello podemos hacer clic en "Opciones avanzadas" y seleccionar "Continuar a empresa1.local (no seguro)".

Actividad 12. Comparación HTTP vs HTTPS

- **HTTP** Puerto 80 Sin cifrado
- **HTTPS** Puerto 443 Cifrado SSL/TLS

Diagrama

- **HTTP** Cliente → Texto plano → Servidor
- **HTTPS** Cliente → SSL/TLS → Cifrado → Servidor

Conclusiones

Al finalizar la práctica se ha comprobado que:

- Apache puede servir contenido mediante HTTPS.
- El protocolo HTTPS utiliza el puerto 443.
- SSL/TLS proporciona cifrado de las comunicaciones.
- Los certificados permiten identificar servidores web.
- Un certificado autofirmado es válido para entornos de laboratorio y pruebas.

Se generó un **certificado SSL/TLS autofirmado** mediante **OpenSSL** y se configuró **Apache** para servir la aplicación web a través del protocolo **HTTPS**. Se verificó el funcionamiento accediendo a la URL **https://empresa1.local**, comprobando que el servidor escucha en el **puerto 443** y que el certificado emitido para el dominio **empresa1.local** es presentado correctamente al navegador. Aunque el navegador indica **"No seguro"** por tratarse de un **certificado autofirmado**, la comunicación se realiza de forma cifrada utilizando **SSL/TLS**.